

JobGen Backend

Repo Onboarding Guide

⚠️ READ THIS BEFORE WRITING ANY CODE

One repo, five independently-runnable Django services, talking only through Kafka events — never direct function calls between services. If you find yourself wanting to `import` across service folders, stop: that belongs in `shared/` instead.

Dev 1 · api-gateway

Dev 2 · cv-parser

Dev 3 · matching-engine

Dev 4 · job-ingestion

Dev 5 · interview-prep + infra

Table of Contents

01	The Big Picture	3
02	Root-Level Files	4
03	Every Service's Shape	5
04	Dev 1 — api-gateway	6
05	Dev 2 — cv-parser	7
06	Dev 3 — matching-engine	8
07	Dev 4 — job-ingestion	9
08	Dev 5 — interview-prep + shared infra	10
09	Rules Everyone Follows	11
10	First Week Checklist	12

01 — The Big Picture

This is **one repo, five independently-runnable Django services**, all talking to each other only through Kafka events (never direct function calls between services). Each service is its own Django project — its own `manage.py`, own dependencies, own Dockerfile. You will spend 95% of your time inside your own service's folder.

```
jobgen-backend/
├─ docker-compose.yml      + spins up everything together (Kafka, Redis, Postgres, all 5 services)
├─ .env.example            + template for required environment variables
├─ CONTRIBUTING.md         + branching/PR rules — read this before your first commit
├─
├─ shared/events/          + THE CONTRACT. Everyone reads this, nobody owns it alone.
├─ infra/                  + Kafka topic config, Postgres init scripts
├─ services/               + your actual code lives here, one folder per service
└─ docs/architecture.md    + the diagrams + explanations from our design sessions
```

THE SINGLE MOST IMPORTANT RULE OF THIS REPO

A service **never** imports code from another service's folder, and **never** calls another service directly. The only way services talk to each other is by publishing/consuming events defined in `shared/events/`. If you find yourself wanting to `import` something from `services/matching-engine/` while working in `services/cv-parser/`, stop — that means something belongs in `shared/` instead, or you're solving the problem the wrong way.



02 — Root-Level Files

What every top-level file and folder is for, and who's responsible for it.

File / Folder	Purpose	Who Touches It
<code>docker-compose.yml</code>	One command (<code>docker-compose up</code>) starts Kafka, Redis, Postgres, and all 5 services together, wired to talk to each other. This is how you test the whole system, not just your one service.	Everyone runs it; infra owner maintains it
<code>.env.example</code>	Lists every environment variable the system needs (DB URLs, Kafka broker address, API keys) with placeholder values. Copy to <code>.env</code> locally and fill in real values — <code>.env</code> is gitignored, never commit real secrets.	Everyone, on first setup
<code>CONTRIBUTING.md</code>	Branch naming (<code>service/<your-service>/<feature></code>), PR rules, commit conventions.	Everyone, read once
<code>shared/events/*.schema.json</code>	The exact, agreed-upon shape of every event (<code>cv.parsed</code> , <code>job.ingested</code> , <code>match.found</code>). This is your contract — changing a field here changes what other services expect.	Publisher proposes changes; consumers review
<code>shared/events/README.md</code>	Plain-English doc listing every event: what it means, who publishes it, who consumes it. Check this first if unsure what an event should contain.	Everyone keeps it updated
<code>infra/kafka/</code>	Kafka topic definitions and broker configuration used by <code>docker-compose</code> .	Infra owner
<code>infra/postgres/</code>	Any database init scripts (e.g. creating separate schemas per service).	Infra owner
<code>docs/architecture.md</code>	The system diagrams and explanations from design sessions — HLD, event flows, ER diagrams.	Everyone reads; update as design evolves

03 — Every Service Follows the Same Internal Shape

This is deliberate: once you understand one service's folder layout, you can navigate any other service immediately.

```
services/<service-name>/
├─ manage.py           + Django's entry point — run this service standalone
├─ requirements.txt     + this service's own Python dependencies
├─ Dockerfile          + how this service gets built into a container
├─ config/             + Django project settings (not your business logic)
│   └─ settings.py
│   └─ urls.py
│   └─ wsgi.py
└─ <app-name>/         + your actual business logic lives here
    └─ models.py        + this service's database tables
    └─ views.py         + REST endpoints (if this service exposes any)
    └─ consumers/       + code that reacts to incoming Kafka events
    └─ publishers/      + code that sends outgoing Kafka events
    └─ (service-specific folders — detailed per dev below)
```

CONFIG/ VS. THE APP FOLDER

`config/` is Django boilerplate (settings, URL routing config, WSGI entrypoint) — you'll touch it rarely, mostly when adding a new environment variable or registering a new URL. The app folder (`gateway/` , `parser/` , `matching/` , `ingestion/` , `prep/`) is where your actual work happens.

1 api-gateway

The single entry point for the frontend — auth, routing, JWTs

Handles login/signup, issues JWTs, routes requests to the right internal service, and publishes commands onto Kafka where needed.

```
services/api-gateway/  
└─ gateway/  
  ├── views.py      ← REST endpoints the frontend calls directly (auth, routing)  
  ├── auth/         ← JWT issuing and validation logic — this is where login/signup lives  
  ├── middleware/    ← rate limiting, request validation — runs before a request reaches views.py  
  └─ clients/        ← small HTTP client functions for calling other services' internal APIs  
                      (e.g. a function that calls matching-engine's GET /internal/matches/{cv_id})
```

WHAT YOU NEED TO KNOW

- You own **auth for the whole system** — every other service trusts that a request reaching it already passed through your JWT validation. Get this right first; everyone else builds on top of it.
- `clients/` is how you call other services' REST APIs (like `matching-engine`'s two `GET` endpoints) — these are just HTTP calls to another container's URL, defined in `docker-compose.yml` and your `.env`.
- You don't touch Kafka much directly for now, since your role is mostly synchronous request routing — but if `cv-parser` or `job-ingestion` ever need a command published from the gateway side, that logic goes here too.
- Read `shared/events/README.md` to know what each service's REST API expects — don't guess at request/response shapes.

2

cv-parser

Extracts structured fields from CVs, generates suggestions

Takes an uploaded CV, extracts structured fields, generates optimization suggestions, and publishes `cv.parsed`.

```
services/cv-parser/
└─ parser/
   │ models.py           ← this service's own CV record (raw file info, parse status)
   │ consumers/          ← empty for now — this service doesn't consume events yet, kept for symmetry
   │ publishers/
   │   └─ cv_parsed.py   ← builds and sends the cv.parsed event once parsing finishes
   │ extraction/         ← the actual PDF/DOCX text-extraction logic (pdfplumber, python-docx)
   │ suggestions/        ← logic that generates "add this keyword" / "quantify this achievement" suggestions
   └─ tasks.py           ← Celery tasks — parsing runs here, asynchronously, not inline in a request
```

WHAT YOU NEED TO KNOW

- **Parsing must be async.** A user uploads a CV, gets an immediate "processing..." response, and the real extraction work happens in a Celery task (`tasks.py`), not inside the HTTP request itself — otherwise a slow parse blocks the whole request.
- **Celery here is different from Kafka.** Kafka is how *services* talk to each other. Celery (with Redis as its broker) is how *this one service* runs a slow background job internally. Don't confuse the two — you'll use both.
- `publishers/cv_parsed.py` only fires **after** extraction fully succeeds — check `shared/events/cv_parsed.schema.json` for the exact payload shape before writing this.
- If a CV fails to parse (bad format, corrupted file), don't crash — flag it and let the user know, per the system's "never fail silently" rule.

3 matching-engine

The algorithmic core — skills, gates, scoring

The algorithmic core — resolves skills, filters candidates, evaluates gates, computes weighted scores, stores matches, and publishes `match.found`.

```
services/matching-engine/
└─ matching/
  │ └─ models.py           ← CV (read model), Job (read model), Match, Skill, SkillAlias
  │ └─ views.py           ← GET /internal/matches/{cv_id}, GET /internal/matches/{cv_id}/{job_id}
  │ └─ consumers/
  │   │ └─ cv_parsed_consumer.py    ← reacts to cv.parsed
  │   │ └─ job_ingested_consumer.py ← reacts to job.ingested
  │   └─ publishers/
  │     │ └─ match_found.py ← builds and sends match.found, once per qualifying candidate
  │     └─ skills/
  │       │ └─ normalize.py ← lowercase, split compound skills, strip punctuation
  │       │ └─ alias_resolver.py ← Redis-backed exact lookup (alias -> canonical skill id)
  │       │ └─ fuzzy_match.py ← difflib-based fallback for typos/near-misses
  │       └─ scoring/
  │         │ └─ gate.py ← required-skills / min-experience / location pass-fail check
  │         │ └─ weighted_score.py ← 0.5 skill + 0.3 experience + 0.2 seniority formula
  │         └─ reconciliation.py ← the CV-edit cleanup: snapshot old matches, diff, delete stale ones
```

WHAT YOU NEED TO KNOW

- This is the busiest service — you have two consumers (one per event type) that both funnel into the **same** `skills/` and `scoring/` logic. Don't duplicate scoring logic per-consumer; both consumers should call the same shared functions.
- **Resolve-all-then-write-once:** in a consumer, resolve every skill string in memory first (via `skills/`), and only write to `models.py` once the full list is ready — never write partially.
- **Upsert everywhere:** every write to `Job`, `CV`, or `Match` uses `update_or_create`, never `create()` — this is what makes your consumers idempotent against Kafka redelivery.
- `reconciliation.py` only runs on the `cv_parsed_consumer` path, specifically when the CV already existed (an edit, not a first-time upload) — see [docs/architecture.md](#) for the full before/after diagram.
- Your `views.py` is **read-only** — it only ever queries `models.py`, never triggers scoring. All scoring happens in the consumers.

Pulls job listings from external sources on a schedule, normalizes them, dedupes, and publishes `job.ingested`.

```
services/job-ingestion/
└─ ingestion/
    │ └─ models.py          ← this service's own record of ingested jobs (for dedup tracking)
    │ └─ publishers/
    │ │ └─ job_ingested.py ← builds and sends the job.ingested event
    │ └─ adapters/         ← one file per external source — each normalizes that source's
    │ │ └─ indeed.py       response shape into your common internal job format
    │ │ └─ remoteok.py
    │ │ └─ upwork.py
    │ └─ dedup.py          ← detects the same job appearing across multiple sources
    └─ tasks.py            ← Celery Beat scheduled jobs — this is what triggers fetching, on a timer
```

WHAT YOU NEED TO KNOW

- **Adapter pattern:** each file in `adapters/` takes that source's raw API/scrape response and returns the *same* normalized shape — so the rest of your pipeline (dedup, publishing) never needs to know which source a job came from.
- `tasks.py` uses **Celery Beat** (scheduled tasks), not a regular Celery task triggered by a request — this runs on a timer (e.g. every few hours), not in response to any user action.
- Respect each source's rate limits in your adapters — getting rate-limited or banned takes down ingestion for everyone.
- If one source's adapter fails (site down, API error), don't let it block the others — wrap each adapter call so one failure doesn't stop the whole scheduled run.

SOURCE_URL MUST BE REAL

`source_url` must be the real, direct link to the listing — that's literally what the frontend's "Apply" button uses, so double check each adapter captures a stable URL, not something that expires.

5 interview-prep

Mock interview generation — plus you own the shared plumbing

Generates mock interview questions and feedback based on matched jobs. **You also own the shared plumbing everyone else depends on.**

```
services/interview-prep/
└─ prep/
   │ └─ models.py
   │ └─ consumers/
   │   │ └─ match_found_consumer.py  + reacts to match.found, can pre-generate questions
   │   └─ generation/              + LLM prompt/response handling — kept isolated so it's
   │                                   easy to swap providers or add caching later
   │ └─ views.py                    + endpoints for starting a session, submitting an answer
PLUS, you own:
└─ docker-compose.yml              + keep this working as new services/dependencies get added
└─ infra/kafka/                    + topic setup
└─ shared/events/                  + keep schemas and the README accurate as they evolve
```

WHAT YOU NEED TO KNOW

- `interview-prep` itself is the lightest service to build — which is why you're also the one holding together `docker-compose.yml`, Kafka topic config, and the shared event schemas.
- **Get `docker-compose.yml` running with a working Kafka + Redis + Postgres skeleton in week 1**, even before any service has real logic — the other four devs are blocked from testing full event flows until this works.
- When someone proposes a change to a `shared/events/*.schema.json` file, you're a natural person to help review it (since you're touching this folder the most), but it's genuinely a team decision — flag changes in your team chat, don't let them merge silently.
- Isolate LLM calls in `generation/` — don't scatter prompt-building or API calls throughout the service. This makes it easy to add caching (per NFR-I2, avoid regenerating the same questions for common roles) without touching unrelated code.

09 — Rules Everyone Follows, Regardless of Service

- 1 Never import across service folders.** If `matching-engine` needs something from `cv-parser`, that's a sign it should be an event, not a shared import.
- 2 Every write to your service's database uses `update_or_create` (upsert), not `create()`**, if there's any chance the same event could be processed twice. Kafka guarantees at-least-once delivery — assume redelivery will happen.
- 3 Every event you publish must match its schema in `shared/events/` exactly.** If you need a new field, update the schema file first and flag it to whoever consumes that event, before you start relying on it in code.
- 4 Log with a correlation ID** so a request/event can be traced across services during debugging — check `docs/architecture.md` for the agreed format once decided.
- 5 `.env` is never committed.** Only `.env.example` (with placeholder values) goes in git.
- 6 Branch per service, per feature:** `service/<your-service>/<short-feature-name>` (e.g. `service/matching-engine/skill-normalization`) — see `CONTRIBUTING.md` for the full PR process.

10 — First Week Checklist

☐ Clone the repo, copy `.env.example` to `.env`

☐ Read `docs/architecture.md` in full — this is the diagrammed version of everything above

☐ Read `shared/events/README.md` — know what every event contains before you write code that touches one

☐ Get `docker-compose up` running locally (Dev 5 should have this ready first)

☐ Set up your own service's `manage.py runserver` (or Celery worker, if that's your service) running standalone

☐ Confirm you can publish/consume a **test** event through Kafka before building real logic — proves your local setup actually works end to end